MODULE IV:- UNIT IV :- Automation TESTING

Introduction to SELENIUM:-

What is Selenium?

Selenium is an **open-source (free) automated testing suite** used to validate web applications across different browsers and platforms. It isn't just one tool; it's a suite of software, each catering to different testing needs.

The Selenium Ecosystem/Components of Selenium:

1 Selenium IDE

- Record and playback tool.
- Works as a browser extension (mainly Chrome & Firefox).
- No programming knowledge required.
- Used for quick test creation and prototyping.

Best for: Beginners & quick test case creation.

2 Selenium WebDriver

- Core component of Selenium.
- Automates browsers by directly interacting with them.
- Supports multiple programming languages:
  - Java
  - Python
  - C#
  - JavaScript
  - Ruby
- Supports multiple browsers:
  - Chrome
  - Firefox
  - Edge
  - Safari

Best for: Advanced automation & framework development.

3 Selenium Grid

- Used for running tests on multiple machines.
- Supports parallel test execution.
- Enables cross-browser testing.
- Saves execution time.

Best for: Large projects & cross-browser testing.

4) Selenium RC (Deprecated)

- Older version of Selenium.
- Allowed automation using JavaScript injection.

- Now replaced by Selenium WebDriver.

Selenium WEB driver:

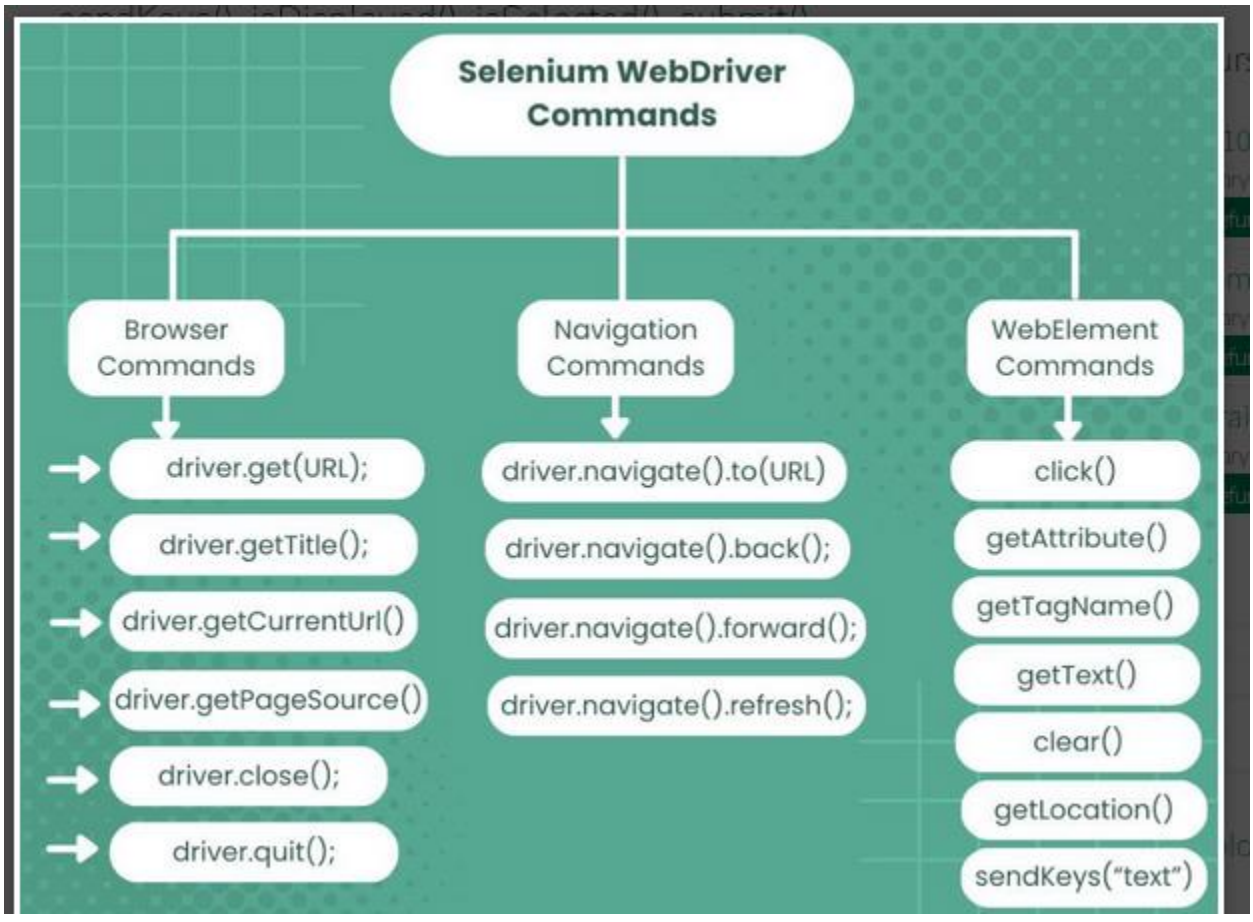
Selenium WebDriver is the core component of Selenium used to automate web browsers.
It directly communicates with the browser through browser-specific drivers like:

- ChromeDriver (for Chrome)
- GeckoDriver (for Firefox)
- EdgeDriver (for Edge)

It supports Java, Python, C#, JavaScript, Ruby, etc.

Important Selenium WebDriver Commands

Below are commonly used WebDriver commands (shown in Java syntax).



1 Browser Commands

Browser commands provide exact control over the browser's actions, like getting specific pages, extracting information such as page titles and URLs, accessing page source code, and controlling browser windows. Browser commands provide an easy way to interact with web applications and to perform automation and scraping data from web pages.

The Browser Command provides methods: get(), getTitle(), getCurrentUrl(), getPageSource() , close() and quit().

A) `get()` **Syntax: driver.get("URL");**

B) `getTitle()` **Syntax: driver.getTitle();**

C) getCurrentUrl() Syntax:- driver.getCurrentUrl();

D) getPageSource() Syntax:- driver.getPageSource();

E) close() Syntax:- driver.close();

F) quit(). Syntax:- driver.quit();

2. Navigation commands:

Navigation commands in Selenium WebDriver perform operations that involve navigating through web pages and controlling browser behaviour. These commands provide an efficient way to manage a browser's history and perform actions like going back and forward between pages and refreshing the current page.

The Navigation Command provides four methods: to(), back(), forward(), and refresh(). These methods allow the WebDriver to perform the following operations:

A)to() Syntax:-driver.navigate().to("URL");

B) back() Syntax:- driver.navigate().back();

C) forward() Syntax:- driver.navigate().forward();

D) refresh(). Syntax:- driver.navigate().refresh();

3. WebElement commands:

To interact with various web element attributes such as (buttons, text fields, links, checkboxes, radio buttons, and more.) selenium webdriver provide a way to interact with all the webelement present on the web page and manipulate them by using Webelement commands. With help of these commands developer or tester can perform various tasks like typing on text field, clicking the button, reading text values, checking and unchecking radio buttons, selecting item from dropdowns etc.

The WebElement Commands provide method: sendKeys(), isDisplayed(), isSelected(), submit(), isEnabled(), getLocation(), clear(), getAttribute(), getText(), getTagName(), click() etc.

a)sendKeys() Syntax:- element.sendKeys("text");

b) isDisplayed() Syntax:- element.isDisplayed();

c) isSelected() Syntax:- boolean selected = element.isSelected();

d) submit() Syntax:- element.submit();

e) isEnabled() Syntax:- boolean enabled = element.isEnabled();

f) **`getLocation()`** Syntax:- Point location = element.getLocation();
System.out.println(location.getX());
System.out.println(location.getY());

g) clear():-Syntax:-element.clear();

h) getAttribute() Syntax:- String value = element.getAttribute("value");

i) getText(), Synatx:- String text = element.getText();

k) getTagName():-Synatx:- String tag = element.getTagName();

l) click() :-Synatx:- element.click();

Locators of Selenium Web driver:-

1. `id`

Finds element by its `id` attribute (fastest & recommended).

driver.findElement(By.id("username"));

2. `name`

Finds element by its `name` attribute.

```
driver.findElement(By.name("email"));
```

🔷 3. `className`

Finds element by its class attribute.

driver.findElement(By.className("loginBtn"));

4. `tagName`

Finds element using HTML tag name.

driver.findElement(By.tagName("input"));

5. `linkText`

Finds hyperlink using exact visible text.

```
driver.findElement(By.linkText("Login"));
```

6. **`partialLinkText`:-** Finds hyperlink using partial visible text.

driver.findElement(By.partialLinkText("Log"));

- **`CSS Selector`**

Uses CSS path to locate elements.

driver.findElement(By.cssSelector("#username"));
driver.findElement(By.cssSelector(".loginBtn"));
driver.findElement(By.cssSelector("input[name='email']"));

8. **`xPath`:-**Uses XML path to locate elements (very powerful).

driver.findElement(By.xpath("//input[@id='username']"));
driver.findElement(By.xpath("//button[text()='Login']"));

TestNG Framework

TestNG (Test Next Generation) is a **Java testing framework** inspired by JUnit and NUnit, mainly used with **Selenium WebDriver** for automation testing.

It is powerful, flexible, and widely used in automation projects.

Why TestNG?

- Supports **Annotations**, Supports **Parallel Execution** ,Supports **Grouping**
- Generates **HTML Reports** ,Supports **Data-driven testing**
- Supports **Test dependencies**,Supports **Parameterization**

Important TestNG Annotations

Annotation	Purpose
`@Test`	Marks a method as test case
`@BeforeMethod`	Runs before each test method
`@AfterMethod`	Runs after each test method
`@BeforeClass`	Runs before first test method in class
`@AfterClass`	Runs after all test methods in class
`@BeforeSuite`	Runs before entire suite
`@AfterSuite`	Runs after entire suite
`@BeforeTest`	Runs before test tag in XML

Annotation	Purpose
`@AfterTest`	Runs after test tag in XML
